Question 1 — Single-cell LSTM forward pass

Given: $x_t = 0.5$, $h_{t-1} = 0.2$, $C_{t-1} = 0.4$. Sigmoid: $\sigma(z) = 1/(1 + e^{-z})$.

Step 1: Forget gate

$z_f = W_f x_t + U_f h_{t-1} + b_f = 0.7(0.5) + 0.3(0.2) + 0.1 = 0.3500 + 0.0600 + 0.1000 = 0.5100$

$f_t = \sigma(0.5100) = 1/(1 + e^{-0.5100}) = 0.6248$

Step 2: Input gate

$z_i = 0.6(0.5) + 0.4(0.2) + 0 = 0.3000 + 0.0800 = 0.3800$

$i_t = \sigma(0.3800) = 1/(1 + e^{-0.3800}) = 0.5939$

Step 3: Candidate cell state

$z_c = 0.9(0.5) + 0.2(0.2) + 0 = 0.4500 + 0.0400 = 0.4900$

$\tilde{C}_t = \tanh(0.4900) = (e^{0.4900} - e^{-0.4900})/(e^{0.4900} + e^{-0.4900}) = 0.4542$

Step 4: Updated cell state

$C_t = f_t C_{t-1} + i_t \tilde{C}_t = (0.6248 \times 0.4) + (0.5939 \times 0.4542)$

Forget contribution = 0.2499; input contribution = 0.2697 (each rounded to 4 decimal places).

$C_t = 0.2499 + 0.2697 = 0.5196$

Step 5: Output gate

$z_o = 0.5(0.5) + 0.5(0.2) + 0.1 = 0.2500 + 0.1000 + 0.1000 = 0.4500$

$o_t = \sigma(0.4500) = 0.6106$

Step 6: Hidden state

$h_t = o_t \times \tanh(C_t) = 0.6106 \times \tanh(0.5196)$

$\tanh(0.5196) = 0.4774$; therefore $h_t = 0.6106 \times 0.4774 = 0.2915$

Answer: $f_t = 0.6248$, $i_t = 0.5939$, $\tilde{C}_t = 0.4542$, $C_t = 0.5196$, $o_t = 0.6106$, $h_t = 0.2915$.

Question 2 — LSTM parameter count

Given: input size d = 10, hidden units h = 32, four gates (forget, input, candidate, output).

Step 1: Parameters per gate

Each gate has an input weight matrix W (h × d), a recurrent weight matrix U (h × h) and a bias vector b (h).

W: 32 × 10 = 320; U: 32 × 32 = 1,024; b: 32

Weights per gate = 320 + 1,024 = 1,344

Step 2: (a) One bias vector per gate

Per gate = 1,344 + 32 = 1,376

Total = 4 × 1,376 = 5,504

Formula check: 4 × [h(d + h) + h] = 4 × [32(42) + 32] = 5,504

Answer: (a) 5,504 trainable parameters.

Step 3: (b) Two bias vectors per gate (PyTorch convention)

Per gate = 1,344 + 32 + 32 = 1,408

Total = 4 × 1,408 = 5,632

This is 4 × 32 = 128 more than part (a), from the extra bias vector in each gate.

Answer: (b) 5,632 trainable parameters.

Question 3 — Vanishing gradient through the cell state

Gradient scaling after k steps with constant forget gate $f \approx f^k$.

Step 1: (a) f = 0.9

k = 10: $0.9^{10} = 0.3487$

k = 50: $0.9^{50} = 0.0052$

Step 2: (b) k = 50 with other forget values

f = 0.5: $0.5^{50} = 8.8818 \times 10^{-16}$ (effectively zero)

f = 0.99: $0.99^{50} = 0.6050$

f	k = 10	k = 50
0.9	0.3487	0.0052
0.5	0.0010	8.8818×10^{-16}
0.99	0.9044	0.6050

Step 3: (c) Interpretation

With f = 0.9, only about 0.5% of the gradient survives 50 steps, and with f = 0.5 it is lost entirely. A dependency 50 steps back can only be learned if the relevant forget gates sit very close to 1 (f = 0.99 keeps about 60%).

A vanilla RNN multiplies the gradient by the recurrent weights and the tanh derivative at every step, so it shrinks (or explodes) exponentially with no control. The LSTM updates its cell state additively, $C_t = f_t C_{t-1} + i_t \tilde{C}_t$, which forms a near-linear gradient path. The forget gate is learned and can stay near 1 where long-term memory is needed, so the gradient is preserved far better.

Answer: (a) 0.3487 (k = 10) and 0.0052 (k = 50); (b) 8.8818×10^{-16} (f = 0.5) and 0.6050 (f = 0.99); (c) forget gates must stay near 1 for long-range learning, and the additive cell-state path lets the LSTM do this.

Question 4 — Electricity load forecasting

Given: 3 years (365 days each) of hourly data; lookback 168 hours; 3 features; forecast horizon 24 hours.

Step 1: (a) Number of hourly records

$N = 3 \times 365 \times 24 = 26{,}280$

Answer: (a) 26,280 records.

Step 2: (b) Windows with stride 1

Each window needs 168 input hours + 24 target hours = 192 consecutive records.

Windows = $N - 192 + 1 = 26{,}280 - 192 + 1 = 26{,}089$

Answer: (b) 26,089 windows.

Step 3: (c) Tensor shapes for a batch of 64

Input: (batch, time steps, features) = (64, 168, 3)

Output: (batch, forecast horizon) = (64, 24)

Answer: (c) Input (64, 168, 3); output (64, 24).

Step 4: (d) Training windows and mini-batches

Training records = $0.70 \times 26{,}280 = 18{,}396$

Training windows = $18{,}396 - 192 + 1 = 18{,}205$ (windows lying entirely within the training segment)

Mini-batches = $\lceil 18{,}205 / 64 \rceil = \lceil 284.45 \rceil = 285$ (284 full batches of 64 = 18,176, plus a final batch of 29)

Answer: (d) 18,205 training windows; 285 mini-batches per epoch.

Step 5: (e) LSTM layer parameters (d = 3, h = 64, single bias)

Per gate = $h(d + h) + h = 64(3 + 64) + 64 = 4{,}288 + 64 = 4{,}352$

Total = $4 \times 4{,}352 = 17{,}408$

Answer: (e) 17,408 trainable parameters.

Question 5 — Predictive maintenance

Given: sensor range 20 to 120 mm/s; 50-cycle LSTM window; binary failure prediction.

Step 1: (a) Min-max normalisation

$x' = (x - x_{min}) / (x_{max} - x_{min}) = (95 - 20) / (120 - 20) = 75 / 100 = 0.7500$

Answer: (a) 0.7500.

Step 2: (b) Prediction at threshold 0.5

$0.83 \geq 0.5$, so the model predicts the positive class (failure).

Answer: (b) Failure predicted.

Step 3: (c) Confusion matrix

Actual failures = 40; flagged = 45; correct flags = 30.

TP = 30

FP = 45 − 30 = 15

FN = 40 − 30 = 10

TN = 1,000 − 30 − 15 − 10 = 945

	Predicted failure	Predicted no failure	Total
Actual failure	TP = 30	FN = 10	40
Actual no failure	FP = 15	TN = 945	960
Total	45	955	1,000

Step 4: (c) Metrics

Precision = TP / (TP + FP) = 30 / 45 = 0.6667

Recall = TP / (TP + FN) = 30 / 40 = 0.7500

F1 = 2 × (0.6667 × 0.7500) / (0.6667 + 0.7500) = 1.0000 / 1.4167 = 0.7059

Accuracy = (TP + TN) / total = (30 + 945) / 1,000 = 0.9750

Answer: (c) TP = 30, FP = 15, FN = 10, TN = 945; precision = 0.6667, recall = 0.7500, F1 = 0.7059, accuracy = 0.9750.

Step 5: (d) Which metric to prioritise

The team should prioritise recall. A missed failure (false negative) means unplanned downtime, possible damage and safety risk, which usually costs far more than a false alarm that only triggers an extra inspection. Precision and F1 still matter so the team is not swamped by false alarms, but recall comes first. Here only 75% of failures are caught, so lowering the decision threshold may be worthwhile.

Accuracy is misleading because the classes are heavily imbalanced: only 4% of cycles are followed by a failure. A model that always predicts "no failure" would score (1,000 − 40) / 1,000 = 0.9600 accuracy while catching no failures (recall = 0). The model's

0.9750 is barely above that useless baseline, so it hides the missed failures.

Answer: (d) Prioritise recall; accuracy is misleading because of class imbalance (a "never fails" model scores 96%).